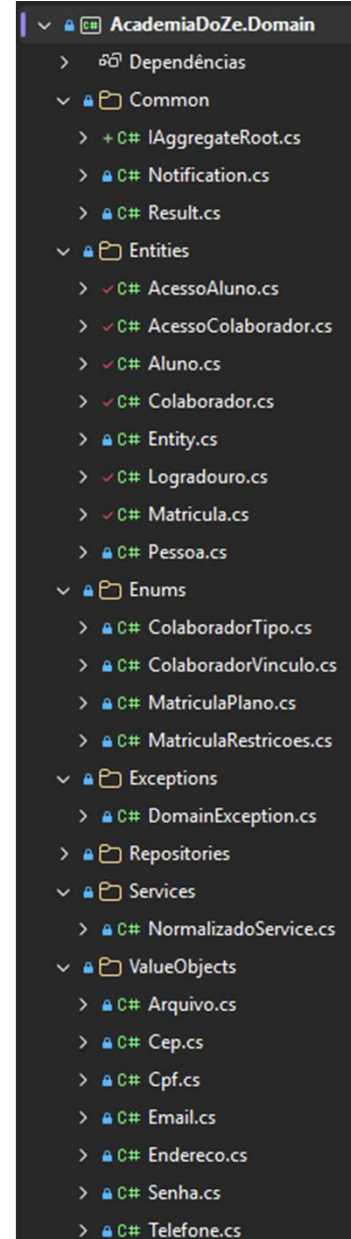


04 de 20

T1 10/08 – T2 13/08

Estrutura de pastas e arquivos

- **Common**
 - Arquivos de uso comum
 - Para notificações de regras de negócio.
- **Entities**
 - Representam objetos do negócio com identidade única.
 - Encapsulam regras e comportamentos.
 - Classes que possuem identidade única, derivando de Entity, e encapsulam o comportamento e consistência do negócio.
 - Posteriormente, a criação de um Aluno, por exemplo, exige o uso de um método estático de fábrica, Criar, que impede que um objeto seja criado em estado inválido, invariantes de domínio.
- **Enums**
 - Usados para estados, tipos, restrições, etc.
 - Com [Flags] se necessário para múltiplas opções.
- **Exception**
 - Para erros de regras de negócio.
- **Services**
 - Fornecem alguns métodos necessários à aplicação, mas que não estão restritos à uma só entidade.
 - Em linhas gerais, contém regras e comportamentos referentes ao modelo de domínio.
 - Serviços sem estado que realizam ações que não pertencem estritamente a uma única entidade. Exemplo: NormalizadoService.
- **Value Objects**
 - Sem identidade, definidos apenas por seus valores.
 - Imutáveis e encapsulam validações.
 - Objetos imutáveis definidos por seus atributos, sem uma identidade única.
 - Exemplo: Cpf, implementado como um record em C# para garantir semântica de igualdade baseada em valores.



- Vejam como ficou a versão final da nossa camada de domínio

AcademiaDoZe.Domain\Services\NormalizacaoService.cs

```
using System.Text.RegularExpressions;

namespace AcademiaDoZe.Domain.Services;

public static partial class NormalizacaoService
{
    // verifica se o texto é nulo ou vazio
    public static bool TextoVazioOuNulo(string? texto) => string.IsNullOrEmpty(texto);

    // remove espaços repetidos e espaços no início e no final do texto
    public static string LimparEspacos(string? texto) => string.IsNullOrEmpty(texto) ? string.Empty : EspacosRegex().Replace(texto, " ").Trim();

    // limpa todos os espaços
    public static string LimparTodosEspacos(string? texto) => string.IsNullOrEmpty(texto) ? string.Empty : texto.Replace(" ", string.Empty);

    // converte o texto para maiúsculo
    public static string ParaMaiusculo(string? texto) => string.IsNullOrEmpty(texto) ? string.Empty : texto.ToUpperInvariant();

    // manter somente digitos numericos
    public static string LimparEDigitos(string? texto) => string.IsNullOrEmpty(texto) ? string.Empty : new string[.. texto.Where(char.IsDigit)];

    [GeneratedRegex(@"\s+")]
    private static partial Regex EspacosRegex();
}
```

AcademiaDoZe.Domain\Exceptions\DomainException.cs

```
namespace AcademiaDoZe.Domain.Exceptions;

// classe base para exceções de domínio
// permitindo exceções específicas de regras de negócio
// uso de construtor primário para simplificar a criação de exceções com mensagem
// sealed para evitar herança adicional, mantendo a hierarquia de exceções clara
public sealed class DomainException(string message) : Exception(message)
{
}
```

AcademiaDoZe.Domain\Common\Notification.cs

```
namespace AcademiaDoZe.Domain.Common;

public record Notification(string Propriedade, string Mensagem);
```

AcademiaDoZe.Domain\Common\Result.cs

```
namespace AcademiaDoZe.Domain.Common;

public class Result<T>
{
    public T? Value { get; }
    public IReadOnlyCollection<Notification> Notifications { get; }
    public bool IsSuccess => Notifications.Count == 0;
    public bool IsFailure => Notifications.Count != 0;

    private Result(T? value, IEnumerable<Notification> notifications)
    {
        Value = value;
        Notifications = notifications.ToList().AsReadOnly();
    }

    public static Result<T> Success(T value) => new(value, []);
    public static Result<T> Failure(IEnumerable<Notification> notifications) => new(default, notifications);
    public static Result<T> Failure(string propriedade, string mensagem) => new(default, [new Notification(propriedade, mensagem)]);
    public static Result<T> Failure(Notification notification) => new(default, [notification]);
}
```

AcademiaDoZe.Domain\Enums\ColaboradorTipo.cs

```
namespace AcademiaDoZe.Domain.Enums;

public enum ColaboradorTipo
{
    Administrador = 0,
    Atendente = 1,
    Instrutor = 2
}
```

AcademiaDoZe.Domain\Enums\MatriculaPlano.cs

```
namespace AcademiaDoZe.Domain.Enums;

public enum MatriculaPlano
{
    Mensal = 0,
    Trimestral = 1,
    Semestral = 2,
    Anual = 3
}
```

AcademiaDoZe.Domain\Enums\ColaboradorVinculo.cs

```
namespace AcademiaDoZe.Domain.Enums;

public enum ColaboradorVinculo
{
    CLT = 0,
    Estagio = 1
}
```

AcademiaDoZe.Domain\Enums\MatriculaRestricoes.cs

```
namespace AcademiaDoZe.Domain.Enums;

[Flags]
public enum MatriculaRestricoes
{
    None = 0,
    Diabetes = 1,
    PressaoAlta = 2,
    Labirintite = 4,
    Alergias = 8,
    ProblemasRespiratorios = 16,
    RemedioContinuo = 32
}
```

AcademiaDoZe.Domain\ValueObjects\Arquivo.cs

```
using AcademiaDoZe.Domain.Common;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Arquivo
{
    public byte[] Conteudo { get; }

    private Arquivo(byte[] conteudo)
    {
        Conteudo = conteudo;
    }

    public static Result<Arquivo> Criar(byte[] conteudo)
    {
        if (conteudo == null)
            return Result<Arquivo>.Failure("Arquivo", "ARQUIVO_OBRIGATORIO");

        const int tamanhoMaximoBytes = 15 * 1024 * 1024; // 15MB
        if (conteudo.Length > tamanhoMaximoBytes)
            return Result<Arquivo>.Failure("Arquivo", "ARQUIVO_TIPO_TAMANHO");

        // cria e retorna o objeto
        return Result<Arquivo>.Success(new Arquivo(conteudo));
    }
}
```

AcademiaDoZe.Domain\ValueObjects\Cep.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Cep
{
    public string Valor { get; }

    private Cep(string valor)
    {
        Valor = valor;
    }

    public static Result<Cep> Criar(string valor)
    {
        if (NormalizacaoService.TextoVazioOuNulo(valor))
            return Result<Cep>.Failure("Cep", "CEP_OBRIGATORIO");

        var textolimpo = NormalizacaoService.LimparEDigitos(valor);
        if (textolimpo.Length != 8)
            return Result<Cep>.Failure("Cep", "CEP_DIGITOS");

        return Result<Cep>.Success(new Cep(textolimpo));
    }

    public override string ToString() => Valor;
}
```


- No Cpf simplifiquei a validação, deixando a entrada dos dados um pouco mais flexível durante nossos testes.
- Ou seja, permitindo que seja possível cadastrar vários Alunos ou Colaboradores sem a necessidade de ter uma lista de CPFs válidos.

```

using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Cpf
{
    public string Valor { get; }

    private Cpf(string valor)
    {
        Valor = valor;
    }

    public static Result<Cpf> Criar(string valor)
    {
        if (NormalizacaoService.TextoVazioOuNulo(valor))
            return Result<Cpf>.Failure("Cpf", "CPF_OBRIGATORIO");

        var textoLimpo = NormalizacaoService.LimparEDigitos(valor);
        if (textoLimpo.Length != 11)
            return Result<Cpf>.Failure("Cpf", "CPF_DIGITOS");

        if (!Validar(textoLimpo))
            return Result<Cpf>.Failure("Cpf", "CPF_INVALIDO");

        return Result<Cpf>.Success(new Cpf(textoLimpo));
    }

    private static bool Validar(string cpf)
    {
        if (cpf.Length != 11) return false;

        /*
        string[] invalidos = ["00000000000", "11111111111", "22222222222", "33333333333", "44444444444", "55555555555", "66666666666", "77777777777", "88888888888", "99999999999"];
        if (invalidos.Contains(cpf)) return false;
        */

        var tempCpf = cpf[..9];
        var soma = 0;

        int[] multiplicador1 = [10, 9, 8, 7, 6, 5, 4, 3, 2];
        for (var i = 0; i < 9; i++)
            soma += (tempCpf[i] - '0') * multiplicador1[i];

        var resto = soma % 11;
        if (resto < 2)
            resto = 0;
        else
            resto = 11 - resto;

        var digito = resto.ToString();
        tempCpf += digito;
        soma = 0;

        int[] multiplicador2 = [11, 10, 9, 8, 7, 6, 5, 4, 3, 2];
        for (var i = 0; i < 10; i++)
            soma += (tempCpf[i] - '0') * multiplicador2[i];

        resto = soma % 11;
        if (resto < 2)
            resto = 0;
        else
            resto = 11 - resto;

        digito += resto.ToString();
        return cpf.EndsWith(digito);
        /*
        return true;
        */
    }

    public override string ToString() => Valor;
}

```

AcademiaDoZe.Domain\ValueObjects\Email.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Email
{
    public string Valor { get; }

    private Email(string valor)
    {
        Valor = valor;
    }

    public static Result<Email> Criar(string valor)
    {
        var textoLimpo = NormalizacaoService.LimparEspacos(valor);

        if (string.IsNullOrEmpty(textoLimpo) || !ValidarFormato(textoLimpo))
            return Result<Email>.Failure("Email", "EMAIL_FORMATO");

        return Result<Email>.Success(new Email(textoLimpo));
    }

    private static bool ValidarFormato(string email)
    {
        var partes = email.Split('@');
        if (partes.Length != 2) return false;
        if (string.IsNullOrEmpty(partes[0])) return false;

        var dominio = partes[1];
        if (string.IsNullOrEmpty(dominio)) return false;
        if (dominio.StartsWith('.') || dominio.EndsWith('.')) return false;

        var labels = dominio.Split('.');
        if (labels.Length < 2) return false;
        if (labels.Any(l => string.IsNullOrEmpty(l))) return false;

        return true;
    }

    public override string ToString() => Valor;
}
```

AcademiaDoZe.Domain\ValueObjects\Senha.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Senha
{
    public string Valor { get; }

    private Senha(string valor)
    {
        Valor = valor;
    }

    public static Result<Senha> Criar(string valor)
    {
        if (NormalizacaoService.TextoVazioOuNulo(valor))
            return Result<Senha>.Failure("Senha", "SENHA_OBRIGATORIO");

        var textoLimpo = NormalizacaoService.LimparEspacos(valor);

        if (textoLimpo.Length < 6 || !textoLimpo.Any(char.IsUpper))
            return Result<Senha>.Failure("Senha", "SENHA_FORMATO");

        return Result<Senha>.Success(new Senha(textoLimpo));
    }

    public override string ToString() => Valor;
}
```

AcademiaDoZe.Domain\ValueObjects\Telefone.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Telefone
{
    public string Valor { get; }

    private Telefone(string valor)
    {
        Valor = valor;
    }

    public static Result<Telefone> Criar(string valor)
    {
        if (NormalizacaoService.TextoVazioOuNulo(valor))
            return Result<Telefone>.Failure("Telefone", "TELEFONE_OBRIGATORIO");

        var textoLimpo = NormalizacaoService.LimparEDigitos(valor);
        if (textoLimpo.Length != 11)
            return Result<Telefone>.Failure("Telefone", "TELEFONE_DIGITOS");

        return Result<Telefone>.Success(new Telefone(textoLimpo));
    }

    public override string ToString() => Valor;
}
```

AcademiaDoZe.Domain\ValueObjects\Endereco.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.ValueObjects;

public record Endereco
{
    public int LogradouroId { get; }
    public string Numero { get; }
    public string Complemento { get; }

    private Endereco(int logradouroId, string numero, string complemento)
    {
        LogradouroId = logradouroId;
        Numero = numero;
        Complemento = complemento;
    }

    public static Result<Endereco> Criar(Logradouro logradouro, string numero, string complemento)
    {
        var notifications = new List<Notification>();

        if (logradouro == null)
            notifications.Add(new Notification("Endereco", "LOGRADOURO_OBRIGATORIO"));

        if (NormalizacaoService.TextoVazioOuNulo(numero))
            notifications.Add(new Notification("Numero", "NUMERO_OBRIGATORIO"));
        else
            numero = NormalizacaoService.LimparEspacos(numero);

        complemento = NormalizacaoService.LimparEspacos(complemento);

        if (notifications.Count != 0)
            return Result<Endereco>.Failure(notifications);

        return Result<Endereco>.Success(new Endereco(logradouro!.Id, numero, complemento));
    }
}
```

- Para versão final de nossas entidades, precisamos:
 - Definir quem são os agregados raiz
 - Preservar suas invariantes
 - Garantir que agregados referenciem outros agregados apenas por identidade

AcademiaDoZe.Domain\Entities\Entity.cs

```
using AcademiaDoZe.Domain.Exceptions;

namespace AcademiaDoZe.Domain.Entities;

// Classe base para todas as entidades, garantindo identidade única e validação de Id
public abstract class Entity
{
    public int Id { get; protected set; }

    protected Entity(int id = 0)
    {
        if (id < 0)
            throw new DomainException("ID_NEGATIVO");

        Id = id;
    }
}
```

AcademiaDoZe.Domain\Entities\Pessoa.cs

```
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Entities;

public abstract class Pessoa : Entity
{
    public string Nome { get; protected set; }
    public Cpf Cpf { get; protected set; }
    public DateOnly DataNascimento { get; protected set; }
    public Telefone Telefone { get; protected set; }
    public Email Email { get; protected set; }
    public Endereco Endereco { get; protected set; }
    public Senha Senha { get; protected set; }
    public Arquivo Foto { get; protected set; }

    protected Pessoa(
        int id,
        string nome,
        Cpf cpf,
        DateOnly dataNascimento,
        Telefone telefone,
        Email email,
        Endereco endereco,
        Senha senha,
        Arquivo foto) : base(id)
    {
        Nome = nome;
        Cpf = cpf;
        DataNascimento = dataNascimento;
        Telefone = telefone;
        Email = email;
        Endereco = endereco;
        Senha = senha;
        Foto = foto;
    }
}
```

- Um agregado, **Aggregate** é um padrão de modelagem do domínio que define um conjunto de Entidades e Value Objects que pertencem ao mesmo contexto de negócio e devem ser tratados como uma única unidade de consistência e transação.
- Em outras palavras, um agregado estabelece uma fronteira dentro da qual todas as regras de negócio devem permanecer válidas.
- Todo **agregado** possui uma entidade principal chamada **Aggregate Root**, Raiz do Agregado.
- A **Aggregate Root** é a entidade responsável por representar e controlar todo o agregado.
- Ela possui as seguintes responsabilidades:
 - Representar o agregado perante o restante do sistema;
 - Garantir que todas as regras de negócio sejam respeitadas;
 - Controlar o acesso às entidades internas;
 - Manter a consistência de todo o agregado.
- Uma regra fundamental do DDD é que:
 - Nenhuma entidade interna pode ser acessada ou modificada diretamente por objetos externos.
 - Toda alteração deve ocorrer por meio da Aggregate Root.
- Dessa forma, a raiz protege o estado interno do agregado, evitando inconsistências e garantindo que todas as validações sejam executadas.

- A ideia é que o **modelo de domínio esteja livre de qualquer definição de infraestrutura de dados**, fazendo com que ele siga os princípios:
 - **POCO** - Plain Old Common-Runtime Object (Objeto comum de tempo de execução padrão)
 - **PI** - Persistence Ignorance (Indiferente da persistência)
- Termos que são utilizados em conjunto para indicar objetos que não possuem comportamentos.
- Os **agregados raiz** são as entidades que controlam o ciclo de vida de suas dependências e garantem a consistência do agregado, em nosso domínio temos o seguinte:
- **Aluno** e **Colaborador** podem ser agregados raiz, pois possuem regras e relacionamentos próprios.
- Para Matrícula, temos duas abordagens, ambas corretas e dependendo a linha de implementação adotada:
 - **Matricula** como agregada do **Aluno**, pois Matrícula é uma entidade que faz parte do ciclo de vida do Aluno, ou seja, ela existe em função do Aluno. Todas as operações que alteram uma Matrícula, criar, atualizar, remover podem ser feitas através do Aluno, garantindo que as regras de negócio do Aluno sejam respeitadas.
 - **Matricula** como agregado raiz, possuindo suas próprias regras e relacionamentos.

- Um dos principais objetivos de um agregado é preservar suas **invariantes**.
- Uma invariante é uma regra de negócio ou condição que deve permanecer verdadeira durante todo o ciclo de vida do agregado.
- Em termos simples:
 - Uma invariante é uma regra que nunca pode ser violada.
- Sempre que uma operação alterar o estado do agregado, a **aggregate root** deve garantir que todas as **invariantes** continuem válidas.
- Exemplos de invariantes:
 - Matrícula
 - A data de término do plano deve ser obrigatoriamente posterior à data de início.
 - Um aluno não pode possuir duas matrículas ativas para o mesmo curso e período.
 - Não é permitido cancelar uma matrícula já concluída.
 - Contexto de Aluno
 - Um aluno não pode possuir dois CPF cadastrados.
 - O CPF deve ser único no sistema.
 - Um aluno não pode ser excluído caso possua matrículas.

- As **entidades internas** que fazem parte do agregado existem apenas dentro de sua fronteira.
- Elas podem possuir identidade própria, porém seu ciclo de vida é controlado exclusivamente pela **Aggregate Root**.
- Da mesma forma, os **Value Objects** pertencentes ao agregado também são manipulados somente através da raiz.
- Outra regra importante do DDD, que utilizaremos em breve, é:
 - **Somente Aggregate Roots possuem Repositórios.**
- O objetivo de um repositório é recuperar e persistir um agregado completo.
- Portanto, não faz sentido criar repositórios para entidades internas, pois elas não devem existir isoladamente do seu agregado.
- Cada Aggregate Root possui seu próprio repositório.

- Precisamos criar uma interface, que servira para **identificar quais entidades atuam como raiz de agregado**.
- Repositórios de domínio só devem persistir entidades que sejam raízes de agregados, por exemplo:
 - Logradouro
 - Aluno
 - Colaborador
 - Matricula
 - AcessoAluno
 - AcessoColaborador

AcademiaDoZe.Domain\Common\IAggregateRoot.cs

```
namespace AcademiaDoZe.Domain.Common;

/// <summary>
/// Interface marcadora para identificar entidades que atuam como Raiz de Agregado (Aggregate Root) no DDD.
/// Repositórios de domínio devem persistir apenas Raízes de Agregados.
/// </summary>
public interface IAggregateRoot
{
}
```

- Agora que já possuímos uma interface marcadora, **IAggregateRoot**, para identificar entidades que atuam como raiz de agregado, basta usar a interface para definir que são os agregados e sua raiz.
- Como estaremos trabalhando nos agregador raiz, já temos que tratar suas **invariantes**.
- E também garantir que agregados referenciem outros agregados apenas por identidade.

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Entities;

public sealed class Logradouro : Entity, IAggregateRoot
{
    // encapsulamento das propriedades, aplicando imutabilidade
    public Cep Cep { get; }
    public string Nome { get; }
    public string Bairro { get; }
    public string Cidade { get; }
    public string Estado { get; }
    public string Pais { get; }

    private Logradouro(int id, Cep cep, string nome, string bairro, string cidade, string estado, string pais) : base(id)
    {
        Cep = cep;
        Nome = nome;
        Bairro = bairro;
        Cidade = cidade;
        Estado = estado;
        Pais = pais;
    }

    public static Result<Logradouro> Criar(int id, string cep, string nome, string bairro, string cidade, string estado, string pais)
    {
        var notifications = new List<Notification>();

        var cepResult = Cep.Criar(cep);
        if (cepResult.IsFailure) notifications.AddRange(cepResult.Notifications);

        if (NormalizacaoService.TextoVazioOuNulo(nome)) notifications.Add(new Notification("Nome", "NOME_OBRIGATORIO"));
        else nome = NormalizacaoService.LimparEspacos(nome);

        if (NormalizacaoService.TextoVazioOuNulo(bairro)) notifications.Add(new Notification("Bairro", "BAIRRO_OBRIGATORIO"));
        else bairro = NormalizacaoService.LimparEspacos(bairro);

        if (NormalizacaoService.TextoVazioOuNulo(cidade)) notifications.Add(new Notification("Cidade", "CIDADE_OBRIGATORIO"));
        else cidade = NormalizacaoService.LimparEspacos(cidade);

        if (NormalizacaoService.TextoVazioOuNulo(estado)) notifications.Add(new Notification("Estado", "ESTADO_OBRIGATORIO"));
        else estado = NormalizacaoService.ParaMaiusculo(NormalizacaoService.LimparTodosEspacos(estado));

        if (NormalizacaoService.TextoVazioOuNulo(pais)) notifications.Add(new Notification("Pais", "PAIS_OBRIGATORIO"));
        else pais = NormalizacaoService.LimparEspacos(pais);

        if (notifications.Count != 0)
            return Result<Logradouro>.Failure(notifications);

        return Result<Logradouro>.Success(new Logradouro(id, cepResult.Value!, nome, bairro, cidade, estado, pais));
    }
}
```

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Services;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Entities;

public class Aluno : Pessoa, IAggregateRoot
{
    // construtor privado para evitar instância direta
    private Aluno(int id, string nome, Cpf cpf, DateOnly dataNascimento, Telefone telefone, Email email, Endereco endereco, Senha senha, Arquivo foto) : base(id, nome, cpf, dataNascimento, telefone, email, endereco, senha, foto) {}

    // método de fábrica, ponto de entrada para criar um objeto válido
    public static Result<Aluno> Criar(int id, string nome, string cpf, DateOnly dataNascimento, string telefone, string email, Logradouro endereco, string numero, string complemento, string senha, Arquivo foto)
    {
        var notifications = new List<Notification>();

        // Validações e normalizações
        if (NormalizacaoService.TextoVazioOuNulo(nome))
            notifications.Add(new Notification("Nome", "NOME_OBRIGATORIO"));
        else
            nome = NormalizacaoService.LimparEspacos(nome);

        if (dataNascimento == default)
            notifications.Add(new Notification("DataNascimento", "DATA_NASCIMENTO_OBRIGATORIO"));
        else if (dataNascimento > DateOnly.FromDateTime(DateTime.Today.AddYears(-12)))
            notifications.Add(new Notification("DataNascimento", "DATA_NASCIMENTO_MINIMA_INVALIDA"));

        // Instanciação e validação via Value Objects
        var cpfResult = Cpf.Criar(cpf);
        if (cpfResult.IsFailure) notifications.AddRange(cpfResult.Notifications);

        var telefoneResult = Telefone.Criar(telefone);
        if (telefoneResult.IsFailure) notifications.AddRange(telefoneResult.Notifications);

        var emailResult = Email.Criar(email);
        if (emailResult.IsFailure) notifications.AddRange(emailResult.Notifications);

        var senhaResult = Senha.Criar(senha);
        if (senhaResult.IsFailure) notifications.AddRange(senhaResult.Notifications);

        var enderecoResult = Endereco.Criar(endereco, numero, complemento);
        if (enderecoResult.IsFailure) notifications.AddRange(enderecoResult.Notifications);

        if (notifications.Count != 0)
            return Result<Aluno>.Failure(notifications);

        // criação e retorno do objeto
        var aluno = new Aluno(
            id,
            nome,
            cpfResult.Value!,
            dataNascimento,
            telefoneResult.Value!,
            emailResult.Value!,
            enderecoResult.Value!,
            senhaResult.Value!,
            foto
        );

        return Result<Aluno>.Success(aluno);
    }
}
```



```

using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.Services;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Entities;

public class Colaborador : Pessoa, IAggregateRoot
{
    // encapsulamento das propriedades, aplicando imutabilidade
    public DateOnly DataAdmissao { get; private set; }
    public ColaboradorTipo Tipo { get; private set; }
    public ColaboradorVinculo Vinculo { get; private set; }

    // construtor privado para evitar instância direta
    private Colaborador(int id, string nome, Cpf cpf, DateOnly dataNascimento, Telefone telefone, Email email, Endereco endereco, Senha senha, Arquivo foto, DateOnly dataAdmissao,
        ColaboradorTipo tipo, ColaboradorVinculo vinculo) : base(id, nome, cpf, dataNascimento, telefone, email, endereco, senha, foto)
    {
        DataAdmissao = dataAdmissao;
        Tipo = tipo;
        Vinculo = vinculo;
    }

    // método de fábrica, ponto de entrada para criar um objeto válido
    public static Result<Colaborador> Criar(int id, string nome, string cpf, DateOnly dataNascimento, string telefone, string email, Logradouro endereco, string numero, string complemento, string senha, Arquivo foto,
        DateOnly dataAdmissao, ColaboradorTipo tipo, ColaboradorVinculo vinculo)
    {
        var notifications = new List<Notification>();

        if (NormalizacaoService.TextoVazioOuNulo(nome)) notifications.Add(new Notification("Nome", "NOME_OBRIGATORIO"));
        else nome = NormalizacaoService.LimparEspacos(nome);

        if (dataNascimento == default) notifications.Add(new Notification("DataNascimento", "DATA_NASCIMENTO_OBRIGATORIO"));
        else if (dataNascimento > DateOnly.FromDateTime(DateTime.Today.AddYears(-12))) notifications.Add(new Notification("DataNascimento", "DATA_NASCIMENTO_MINIMA_INVALIDA"));

        if (dataAdmissao == default) notifications.Add(new Notification("DataAdmissao", "DATA_ADMISSAO_OBRIGATORIO"));
        else if (dataAdmissao > DateOnly.FromDateTime(DateTime.Today)) notifications.Add(new Notification("DataAdmissao", "DATA_ADMISSAO_MAIOR_QUE_ATUAL"));

        if (!Enum.IsDefined(tipo)) notifications.Add(new Notification("Tipo", "TIPO_COLABORADOR_INVALIDO"));
        if (!Enum.IsDefined(vinculo)) notifications.Add(new Notification("Vinculo", "VINCULO_COLABORADOR_INVALIDO"));

        if (Enum.IsDefined(tipo) && Enum.IsDefined(vinculo) && tipo == ColaboradorTipo.Administrador && vinculo != ColaboradorVinculo.CLT) notifications.Add(new Notification("Vinculo", "ADMINISTRADOR_CLT_INVALIDO"));

        // Instanciã o e valida  o via Value Objects
        var cpfResult = Cpf.Criar(cpf);
        if (cpfResult.IsFailure) notifications.AddRange(cpfResult.Notifications);

        var telefoneResult = Telefone.Criar(telefone);
        if (telefoneResult.IsFailure) notifications.AddRange(telefoneResult.Notifications);

        var emailResult = Email.Criar(email);
        if (emailResult.IsFailure) notifications.AddRange(emailResult.Notifications);

        var senhaResult = Senha.Criar(senha);
        if (senhaResult.IsFailure) notifications.AddRange(senhaResult.Notifications);

        var enderecoResult = Endereco.Criar(endereco, numero, complemento);
        if (enderecoResult.IsFailure) notifications.AddRange(enderecoResult.Notifications);

        if (notifications.Count != 0)
            return Result<Colaborador>.Failure(notifications);

        // cria  o e retorno do objeto
        var colaborador = new Colaborador(id, nome, cpfResult.Value!, dataNascimento, telefoneResult.Value!, emailResult.Value!, enderecoResult.Value!, senhaResult.Value!, foto, dataAdmissao, tipo, vinculo);

        return Result<Colaborador>.Success(colaborador);
    }
}

```

- No **DDD**, **não é o ideal** um Agregado armazenar a referência direta para outro Agregado.
 - **Fronteira de transação e isolamento:**
 - Cada Raiz de Agregado é uma fronteira de consistência.
 - Se Matricula contiver uma instância completa de Aluno, corre-se o risco de modificar o estado de Aluno através de Matricula, violando o limite de transação do agregador.
 - **Performance e carregamento:**
 - Para consultar uma Matricula, o repositório seria forçado a carregar todo o grafo de Aluno, o que gera *queries* desnecessariamente pesadas.
 - **Imutabilidade histórica:**
 - A matrícula pertence ao aluno com determinado Alunold.
 - Se os dados cadastrais do aluno mudarem no futuro, exemplo mudou de endereço ou telefone, isso não deve impactar a estrutura interna da matrícula.
- Apesar de não armazenarmos a referencia direta de Aluno, precisamos dela para poder realizar as validações, ou seja, não teríamos como validar a idade do aluno, com base em somente seu ID.
- Para isso, vamos garantir que Matricula não guarde referencia direta de outros agregados, guardando somente seu Id.
- E usaremos a referencia direta de Aluno no método de fabrica, Criar.
- Dessa forma conseguiremos realizar as validações necessárias, e manteremos Matricula dentro das boas práticas.

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.Services;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Entities;

public class Matricula : Entity, IAggregateRoot
{
    // encapsulamento das propriedades, aplicando imutabilidade
    public int AlunoId { get; private set; }
    public MatriculaPlano Plano { get; private set; }
    public DateOnly DataInicio { get; private set; }
    public DateOnly DataFim { get; private set; }
    public string Objetivo { get; private set; }
    public MatriculaRestricoes RestricoesMedicas { get; private set; }
    public string ObservacoesRestricoes { get; private set; }
    public Arquivo? LaudoMedico { get; private set; }

    // construtor privado para evitar instância direta
    private Matricula(int id, int alunoId, MatriculaPlano plano, DateOnly dataInicio, DateOnly dataFim, string objetivo, MatriculaRestricoes restricoesMedicas, Arquivo? laudoMedico, string observacoesRestricoes = "") : base(id)
    {
        AlunoId = alunoId;
        Plano = plano;
        DataInicio = dataInicio;
        DataFim = dataFim;
        Objetivo = objetivo;
        RestricoesMedicas = restricoesMedicas;
        LaudoMedico = laudoMedico;
        ObservacoesRestricoes = observacoesRestricoes;
    }

    // método de fábrica, ponto de entrada para criar um objeto válido
    public static Result<Matricula> Criar(int id, Aluno aluno, MatriculaPlano plano, DateOnly dataInicio, string objetivo, MatriculaRestricoes restricoesMedicas, Arquivo? laudoMedico, string observacoesRestricoes = "")
    {
        var notifications = new List<Notification>();

        // Validações e normalizações
        if (aluno == null)
        {
            notifications.Add(new Notification("Aluno", "ALUNO_INVALIDO"));
        }
        else if (aluno.DataNascimento > DateOnly.FromDateTime(DateTime.Today.AddYears(-16)) && laudoMedico == null)
        {
            notifications.Add(new Notification("LaudoMedico", "MENOR_16_LAUDO_OBRIGATORIO"));
        }

        if (!Enum.IsDefined(plano)) notifications.Add(new Notification("Plano", "PLANO_INVALIDO"));

        if (dataInicio == default) notifications.Add(new Notification("DataInicio", "DATA_INICIO_OBRIGATORIO"));

        // Cálculo da DataFim no domínio baseado no tipo de plano
        DateOnly dataFim = default;
        if (Enum.IsDefined(plano) && dataInicio != default)
        {
            dataFim = plano switch
            {
                MatriculaPlano.Mensal => dataInicio.AddMonths(1),
                MatriculaPlano.Trimestral => dataInicio.AddMonths(3),
                MatriculaPlano.Semestral => dataInicio.AddMonths(6),
                MatriculaPlano.Anual => dataInicio.AddMonths(12),
                _ => default
            };
        }

        if (NormalizacaoService.TextoVazioOuNulo(objetivo)) notifications.Add(new Notification("Objetivo", "OBJETIVO_OBRIGATORIO"));
        else objetivo = NormalizacaoService.LimparEspacos(objetivo);

        if (restricoesMedicas != MatriculaRestricoes.None && laudoMedico == null) notifications.Add(new Notification("LaudoMedico", "RESTRICOES_LAUDO_OBRIGATORIO"));

        observacoesRestricoes = NormalizacaoService.LimparEspacos(observacoesRestricoes);

        if (notifications.Count != 0) return Result<Matricula>.Failure(notifications);

        // criação e retorno do objeto
        var matricula = new Matricula(id, aluno.Id, plano, dataInicio, dataFim, objetivo, restricoesMedicas, laudoMedico, observacoesRestricoes);

        return Result<Matricula>.Success(matricula);
    }
}
```

- Em Acessos, além do não uso de referencia direta de agregado, também temos casos onde não possuímos meios para realizar as validações, pois dependemos de dados já persistidos.
- Por exemplo:
 - Validar se já não ultrapassa o limite de: 8 horas se for ctl, 6 horas se for estágio.
 - Na saída, mostrar o tempo que permaneceu na academia, devendo ser somado todos os registros do dia.
- Não temos como validar esses dados exclusivamente na camada de domínio, e estes terão que ser tratados em camadas superiores.

AcademiaDoZe.Domain\Entities\AcessoColaborador.cs

```
using AcademiaDoZe.Domain.Common;

namespace AcademiaDoZe.Domain.Entities;

public class AcessoColaborador : Entity, IAggregateRoot
{
    public int ColaboradorId { get; private set; }
    public DateTime DataHora { get; private set; }

    private AcessoColaborador(int id, int colaboradorId, DateTime dataHora) : base(id)
    {
        ColaboradorId = colaboradorId;
        DataHora = dataHora;
    }

    public static Result<AcessoColaborador> Criar(int id, Colaborador colaborador, DateTime dataHora)
    {
        var notifications = new List<Notification>();

        if (colaborador == null)
            notifications.Add(new Notification("Colaborador", "COLABORADOR_INVALIDO"));

        if (dataHora.TimeOfDay < new TimeSpan(6, 0, 0) || dataHora.TimeOfDay > new TimeSpan(22, 0, 0))
            notifications.Add(new Notification("DataHora", "DATA_HORA_INTERVALO_INVALIDO"));

        if (notifications.Count != 0)
            return Result<AcessoColaborador>.Failure(notifications);

        return Result<AcessoColaborador>.Success(new AcessoColaborador(id, colaborador!.Id, dataHora));
    }
}

// Dependem da persistência:
// Validar se já não ultrapassa o limite de: 8 horas se for ctl, 6 horas se for estágio.
// Na saída, mostrar o tempo que permaneceu na academia, devendo ser somado todos os registros do dia.
```

AcademiaDoZe.Domain\Entities\AcessoAluno.cs

```
using AcademiaDoZe.Domain.Common;

namespace AcademiaDoZe.Domain.Entities;

public class AcessoAluno : Entity, IAggregateRoot
{
    public int AlunoId { get; private set; }
    public DateTime DataHora { get; private set; }

    private AcessoAluno(int id, int alunoId, DateTime dataHora) : base(id)
    {
        AlunoId = alunoId;
        DataHora = dataHora;
    }

    public static Result<AcessoAluno> Criar(int id, Aluno aluno, DateTime dataHora)
    {
        var notifications = new List<Notification>();

        if (aluno == null)
            notifications.Add(new Notification("Aluno", "ALUNO_INVALIDO"));

        if (dataHora.TimeOfDay < new TimeSpan(6, 0, 0) || dataHora.TimeOfDay > new TimeSpan(22, 0, 0))
            notifications.Add(new Notification("DataHora", "DATA_HORA_INTERVALO_INVALIDO"));

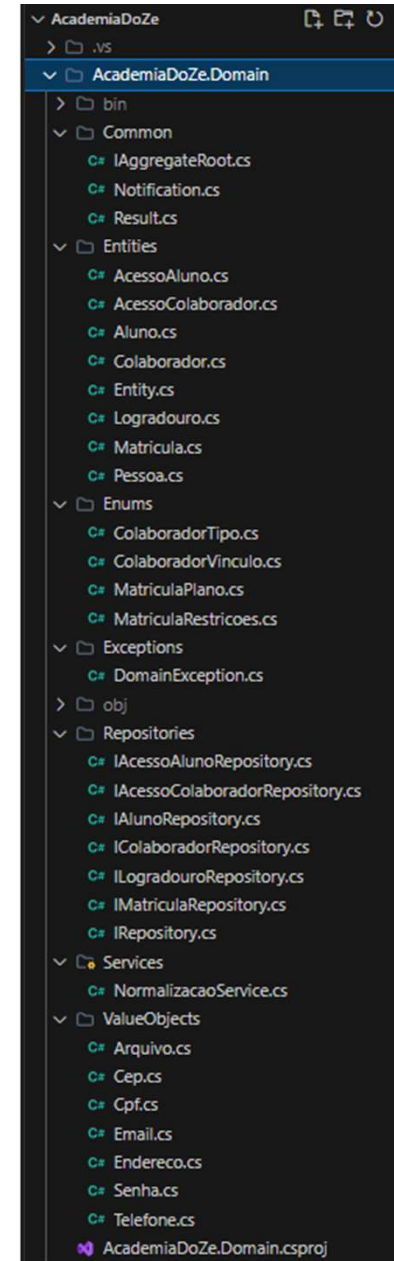
        if (notifications.Count != 0)
            return Result<AcessoAluno>.Failure(notifications);

        return Result<AcessoAluno>.Success(new AcessoAluno(id, aluno!.Id, dataHora));
    }
}

// Dependem da persistência:
// Validar se possui matrícula ativa
// Na entrada, mostrar quanto tempo ainda tem de plano
// Na saída, mostrar o tempo que permaneceu na academia
```

- **Feito!**
- Nossa **camada de domínio está criada!**
- É claro que ainda poderíamos evoluí-la, implementando outros métodos nas entidades e incorporando comportamentos de negócio mais expressivos, por exemplo:
 - Matricula.Renovar()
 - Matricula.EstaAtiva()
 - Aluno.AtualizarDadosPessoais()
 - entre outros.
- Esses comportamentos ajudariam a concentrar as regras de negócio no próprio domínio, tornando o modelo ainda mais rico e expressivo.
- No entanto, precisamos manter o projeto dentro de um **escopo adequado à disciplina**. Por isso, será necessário abrir mão de algumas implementações e simplificar determinados pontos, sem, é claro, deixar de lado os **conceitos e fundamentos essenciais** que estamos estudando.
- O objetivo, neste momento, é construir um sistema completo e pronto para produção, **aplicando corretamente os principais conceitos de DDD e Clean Architecture**, entendendo onde cada responsabilidade deve estar e como as diferentes partes da aplicação se relacionam.

- Agora que já estruturamos a **camada de domínio**, o próximo passo é a criação das **interfaces de repositório**.
- Os **repositórios** têm a responsabilidade de **abstrair o acesso e a persistência das entidades agregadas**, permitindo que o domínio trabalhe com objetos, sem se preocupar com os detalhes da infraestrutura.
- As interfaces de repositório exercem um papel fundamental no DDD e na arquitetura de aplicações modernas, cumprindo as seguintes funções:
 - **Definição de contratos:** Especificam as operações que podem ser realizadas sobre as entidades do domínio, como persistir, buscar, atualizar e remover.
 - **Abstração da persistência:** Permitem que o domínio interaja apenas com contratos, sem depender de tecnologias específicas como bancos de dados, ORMs ou serviços externos.
 - **Facilidade nos testes:** Como o domínio depende apenas das interfaces, é simples criar mocks ou fakes para testes unitários.
 - **Organização do acesso aos dados:** Cada entidade raiz ou agregado importante possui seu próprio repositório, centralizando as regras de acesso e consulta.
- O domínio permanece isolado da infraestrutura, **desacoplado**, resultando em um sistema mais flexível e de fácil manutenção.
- Mudanças na tecnologia de persistência, exemplo troca de banco de dados ou ORM, exigem apenas a substituição da implementação, mantendo os contratos inalterados, proporcionando **facilidade de evolução**.
- Regras de negócio e validações podem ser aplicadas antes da consulta ou persistência dos dados, garantindo **segurança e consistência**.



- Com base nos conceitos de agregados do DDD e na estrutura do domínio, as interfaces de repositório devem ser criadas apenas para os **agregados raiz** e **entidades que precisam ser persistidas diretamente**.
- Para garantir isso, implementamos e utilizamos a interface marcadora **IAggregateRoot**, e utilizamos ela em todos nossos **agregados raiz**.
- Como marcamos nossos agregados raiz, agora na criação dos repositórios, vamos restringir a persistência apenas para quem é raiz, fazendo uso do **where: where TEntity : Entity, IAggregateRoot**.

- Incluiremos **Task** em todas as assinaturas dos nossos métodos contidos nas interfaces.
- **Task** representa uma **operação assíncrona** que pode ser executada em **segundo plano**, permitindo que o programa continue respondendo sem bloquear a thread principal.
- Principais funções da Task
 - **Executar código assíncrono**: Permite que métodos sejam executados de forma não bloqueante.
 - **Controlar concorrência**: Ideal para operações de I/O (como leitura de arquivos ou chamadas a APIs), sem travar a aplicação.
 - **Aguardar resultados com await**: Usada com **async/await** para tornar o código assíncrono mais legível.
- Também faremos uso de **CancellationToken** nas operações assíncronas.
- Isso permite que requisições canceladas pelo usuário ou por timeout interrompam automaticamente as consultas no banco de dados, liberando recursos da infraestrutura imediatamente.

AcademiaDoZe.Domain\Repositories\IRepository.cs

```
using AcademiaDoZe.Domain.Common;
using AcademiaDoZe.Domain.Entities;

namespace AcademiaDoZe.Domain.Repositories;

// Interface genérica para repositórios. Restrita apenas a Raízes de Agregado (Aggregate Roots) no DDD.
// Define os contratos essenciais para a persistência de dados.
// Herda de Entity para garantir que TEntity seja uma entidade válida, e seu uso somente no domain.
// Métodos assíncronos (Task), alinhados com práticas modernas de acesso a dados.
public interface IRepository<TEntity> where TEntity : Entity, IAggregateRoot
{
    Task<TEntity?> ObterPorId(int id, CancellationToken cancellationToken = default);
    Task<IEnumerable<TEntity>> ObterTodos(CancellationToken cancellationToken = default);
    Task<TEntity> Adicionar(TEntity entity, CancellationToken cancellationToken = default);
    Task<TEntity> Atualizar(TEntity entity, CancellationToken cancellationToken = default);
    Task<bool> Remover(int id, CancellationToken cancellationToken = default);
}
```

AcademiaDoZe.Domain\Repositories\ILogradouroRepository.cs

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Repositories;

public interface ILogradouroRepository : IRepository<Logradouro>
{
    // Métodos específicos do domínio
    Task<Logradouro?> ObterPorCep(Cep cep, CancellationToken cancellationToken = default);
    Task<bool> CepJaExiste(Cep cep, int? id = null, CancellationToken cancellationToken = default);
    Task<IEnumerable<Logradouro>> ObterPorCidade(string cidade, CancellationToken cancellationToken = default);
    Task<IEnumerable<Logradouro>> ObterPorBairro(string cidade, string bairro, CancellationToken cancellationToken = default);
}
```

AcademiaDoZe.Domain\Repositories\IColaboradorRepository.cs

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Repositories;

public interface IColaboradorRepository : IRepository<Colaborador>
{
    // Métodos específicos do domínio
    Task<Colaborador?> ObterPorCpf(Cpf cpf, CancellationToken cancellationToken = default);
    Task<Colaborador?> ObterPorEmail(Email email, CancellationToken cancellationToken = default);
    Task<bool> CpfJaExiste(Cpf cpf, int? id = null, CancellationToken cancellationToken = default);
    Task<bool> EmailJaExiste(Email email, int? id = null, CancellationToken cancellationToken = default);
    Task<IEnumerable<Colaborador>> ObterPorTipo(ColaboradorTipo tipo, CancellationToken cancellationToken = default);
    Task<IEnumerable<Colaborador>> ObterPorVinculo(ColaboradorVinculo vinculo, CancellationToken cancellationToken = default);
    Task<bool> TrocarSenha(int id, Senha novaSenha, CancellationToken cancellationToken = default);
}
```

AcademiaDoZe.Domain\Repositories\IALunoRepository.cs

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Repositories;

public interface IALunoRepository : IRepository<Aluno>
{
    // Métodos específicos do domínio
    Task<Aluno?> ObterPorCpf(Cpf cpf, CancellationToken cancellationToken = default);
    Task<Aluno?> ObterPorEmail(Email email, CancellationToken cancellationToken = default);
    Task<bool> CpfJaExiste(Cpf cpf, int? id = null, CancellationToken cancellationToken = default);
    Task<bool> EmailJaExiste(Email email, int? id = null, CancellationToken cancellationToken = default);
    Task<IEnumerable<Aluno>> ObterPorNome(string nome, CancellationToken cancellationToken = default);
    Task<bool> TrocarSenha(int id, Senha novaSenha, CancellationToken cancellationToken = default);
}
```

AcademiaDoZe.Domain\Repositories\IMatriculaRepository.cs

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Enums;

namespace AcademiaDoZe.Domain.Repositories;

public interface IMatriculaRepository : IRepository<Matricula>
{
    // Métodos específicos do domínio
    Task<IEnumerable<Matricula>> ObterPorAluno(int alunoId, CancellationToken cancellationToken = default);
    Task<Matricula?> ObterMatriculaAtivaPorAluno(int alunoId, CancellationToken cancellationToken = default);
    Task<bool> PossuiMatriculaAtiva(int alunoId, CancellationToken cancellationToken = default);
    Task<IEnumerable<Matricula>> ObterAtivas(int alunoId = 0, CancellationToken cancellationToken = default);
    Task<IEnumerable<Matricula>> ObterVencendoEmDias(int dias, CancellationToken cancellationToken = default);
    Task<IEnumerable<Matricula>> ObterPorPlano(MatriculaPlano plano, CancellationToken cancellationToken = default);
}
```

AcademiaDoZe.Domain\Repositories\IAcessoAlunoRepository.cs

```
using AcademiaDoZe.Domain.Entities;

namespace AcademiaDoZe.Domain.Repositories;

public interface IAcessoAlunoRepository : IRepository<AcessoAluno>
{
    Task<IEnumerable<AcessoAluno>> ObterAcessosPorAlunoPeriodo(int? alunoId = null, DateOnly? inicio = null, DateOnly? fim = null, CancellationToken cancellationToken = default);
    Task<AcessoAluno?> ObterUltimoAcesso(int alunoId, CancellationToken cancellationToken = default);
    Task<bool> EstaNaAcademia(int alunoId, CancellationToken cancellationToken = default);
    // horário mensal de maior procura, baseado na entrada, por exemplo, em dezembro o horário de maior procura é entre 18h e 20h
    // Retorna um dicionário onde a chave é o horário e o valor é a quantidade de acessos nesse horário
    Task<Dictionary<TimeOnly, int>> ObterHorarioMaisProcuradoPorMes(int mes, CancellationToken cancellationToken = default);
    // Permanência média dos alunos na academia, mensal.
    // retorna um dicionário onde a chave é o mês e o valor é a média de permanência dos alunos nesse mês
    Task<Dictionary<int, TimeSpan>> ObterPermanenciaMediaPorMes(int mes, CancellationToken cancellationToken = default);
    // alunos que não registraram acesso nos últimos x dias
    Task<IEnumerable<Aluno>> ObterAlunosSemAcessoNosUltimosDias(int dias, CancellationToken cancellationToken = default);
}

// Validar se possui matrícula ativa.
// Na entrada, mostrar quanto tempo ainda tem de plano.
// Na saída, mostrar o tempo que permaneceu na academia.
```


AcademiaDoZe.Domain\Repositories\IAcessoColaboradorRepository.cs

```
using AcademiaDoZe.Domain.Entities;

namespace AcademiaDoZe.Domain.Repositories;

public interface IAcessoColaboradorRepository : IRepository<AcessoColaborador>
{
    Task<IEnumerable<AcessoColaborador>> ObterAcessosPorColaboradorPeriodo(int? colaboradorId = null, DateOnly? inicio = null, DateOnly? fim = null, CancellationToken cancellationToken = default);
    Task<AcessoColaborador?> ObterUltimoAcesso(int colaboradorId, CancellationToken cancellationToken = default);
    Task<TimeSpan> ObterHorasTrabalhadasNoDia(int colaboradorId, DateOnly data, CancellationToken cancellationToken = default);
}
```


- Antes de avançarmos no desenvolvimento de nossa solução, precisamos garantir que nossa **camada de domínio** esta OK.
- Ainda estamos longe de termos uma aplicação funcional, então precisamos de uma alternativa para validarmos nossa camada de domínio, algo que é essencial para garantir que tudo esteja sólido e evitar retrabalho.
- Algumas alternativas para essa validação:
 - Revisão de modelagem de domínio (**Design Review**)
 - Analise do código para garantir que os conceitos do negócio estão corretamente representados.
 - **Peer Review**
 - Peça para outro desenvolvedor revisar o código e levantar dúvidas conceituais, sugestões de simplificação ou alertas sobre decisões perigosas.
 - Validação com exemplos reais (**Testes de Cenários**)
 - Simular casos de uso reais, mesmo sem UI ou persistência.
 - Rodar ferramentas de análise estática
 - Garantir qualidade do código e possíveis falhas estruturais.
 - <https://learn.microsoft.com/pt-br/visualstudio/code-quality/roslyn-analyzers-overview?view=vs-2022>
 - Exemplos: Roslyn Analyzers, SonarQube, ReSharper, StyleCop, Roslynator, xUnit Analyzeers, Sonar Analyzer etc.
 - Cobertura com **Testes Unitários**
 - Garantir o correto funcionamento das regras de negócio.

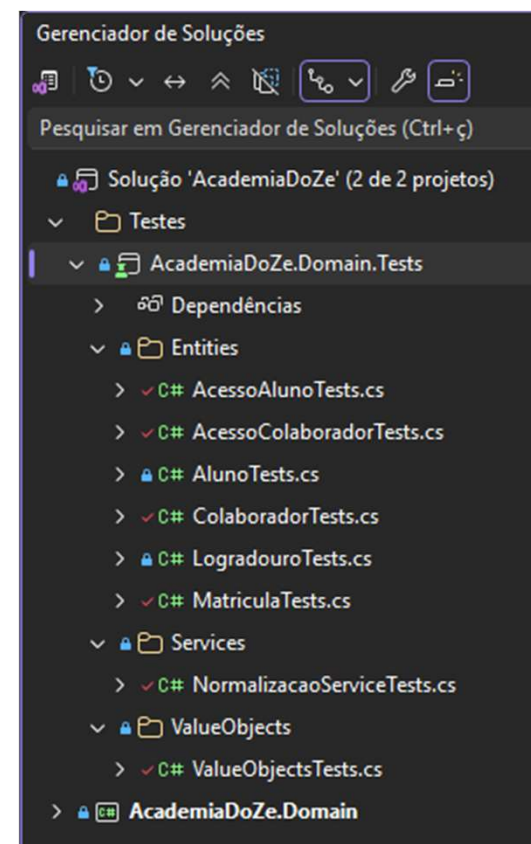


Teste Unitário

- **Teste Unitário** é um teste que verifica a **menor unidade de código possível**, geralmente um método, de forma isolada.
- Isso significa que, podemos realizar testes de forma isolada em nossa **Domain**, sem ter a necessidade/dependência das outras camadas.

Criando o Projeto de Teste

- Na sua solução **AcademiaDoZe** no Visual Studio, clique com o botão direito na Solução, e selecione Adicionar -> Novo Projeto....
- Na barra de busca de templates, digite **xunit**, selecionando o template **Projeto de Testes xUnit**.
 - Também poderíamos utilizar MSTest Test Project, que é outra framework popular.
 - Ambas funcionam de forma semelhante, mas **xUnit** é muito comum em projetos .NET modernos.
- Avance, e nomeie o projeto seguindo uma convenção padrão: [ProjetoTestado].Tests.
- No nosso caso, será **AcademiaDoZe.Domain.Tests**.



Configurando as referências

- O projeto de teste precisa ter acesso ao projeto que ele vai testar.
- No projeto **AcademiaDoZe.Domain.Tests**, clique com o botão direito em **Adicionar -> Referência de Projeto**, e marque a caixa de seleção para **AcademiaDoZe.Domain**.
- Agora, o código no seu projeto de teste pode criar instâncias de classes do projeto Domain.

- Tipos de testes:
 - **[Fact]**
 - Define um **teste unitário simples**, executado uma vez, sem parâmetros.
 - Testa um cenário fixo;
 - **[Theory]**
 - Define um **teste parametrizado**, que será executado várias vezes com diferentes conjuntos de dados.
 - Testa um comportamento com diferentes dados;
 - **[InlineData]**
 - Fornece **valores diretamente para um [Theory]**, criando uma execução do teste para cada conjunto de valores informado.
 - Inlinedata para casos simples.
 - **[MemberData]**
 - Obtém os dados de uma propriedade, campo ou método estático.
 - MemberData ou TheoryData quando os dados forem mais numerosos ou complexos.
 - **[TheoryData<T...>]**
 - Forma fortemente tipada de organizar os dados de um Theory, especialmente útil nas versões mais recentes do xUnit.
 - MemberData ou TheoryData quando os dados forem mais numerosos ou complexos.
 - **[ClassData]**
 - Obtém os dados de uma classe que implementa IEnumerable<object[]>.
 - ClassData quando a fonte de dados justificar uma classe própria

Primeiros testes

- Como exemplo prático, nosso foco inicial serão os ValueObjects, com foco em **Cep** e **Endereco**.
- Criaremos 5 testes unitários, realizando a entrada de vários valores através do [InlineData].
- Desta forma já conseguimos realizar vários testes de uma única vez.
- Renomeie o arquivo UnitTest1.cs, ou crie um novo arquivo, no projeto de testes para algo mais descritivo, como **ValueObjectsTests.cs**.

```

using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.ValueObjects;

public class ValueObjectsTests
{
    [Theory(DisplayName = "Cep: dígitos inválidos -> CEP_DIGITOS")]
    [InlineData("123")]
    [InlineData("12-345")]
    public void Deve_Falhar_Criacao_Quando_CepDigitosInvalidos(string input)
    {
        var result = Cep.Criar(input);

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
    }

    [Theory(DisplayName = "Cep: formatos válidos (com e sem hífen)")]
    [InlineData("12345-678")]
    [InlineData("12345678")]
    public void Deve_Criar_Cep_Quando_Valido(string input)
    {
        var result = Cep.Criar(input);

        Assert.True(result.IsSuccess);
        Assert.Equal("12345678", result.Value!.Valor);
    }

    [Theory(DisplayName = "Cep: obrigatório -> CEP_OBRIGATORIO")]
    [InlineData(null)]
    [InlineData("")]
    public void Deve_Falhar_Criacao_Quando_CepNuloOuVazio(string? input)
    {
        var result = Cep.Criar(input!);

        Assert.True(result.IsFailure);
        Assert.Contains(result.Notifications, n => n.Mensagem == "CEP_OBRIGATORIO");
    }

    [Theory(DisplayName = "Endereco: criação válida com número e complemento")]
    [InlineData("10", "Bloco A")]
    [InlineData("1", "")]
    public void Deve_Criar_Endereco_Quando_Valido(string numero, string complemento)
    {
        var logradouro = Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
        var result = Endereco.Criar(logradouro, numero, complemento);

        Assert.True(result.IsSuccess);
        Assert.Equal(logradouro.Id, result.Value!.LogradouroId);
        Assert.Equal(numero, result.Value.Numero);
        Assert.Equal(complemento, result.Value.Complemento);
    }

    [Theory(DisplayName = "Endereco: valida obrigatoriedade do logradouro e número")]
    [InlineData(null, "1", "LOGRADOURO_OBRIGATORIO")]
    [InlineData("valid", "", "NUMERO_OBRIGATORIO")]
    public void Deve_Falhar_Criacao_Quando_EnderecoInvalido(string logradouroCase, string numero, string expected)
    {
        Logradouro? logradouro = null;
        if (logradouroCase == "valid")
            logradouro = Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;

        var result = Endereco.Criar(logradouro!, numero, "");

        Assert.True(result.IsFailure);
        Assert.Contains(result.Notifications, n => n.Mensagem == expected);
    }
}

```

Executando os testes

- No menu superior do Visual Studio, vá em **Teste**, opção **Gerenciador de Testes**.
- Uma nova janela **Gerenciador de Testes** aparecerá, listando todos os testes encontrados em sua solução.
- Nossos testes criados devem aparecer ali, mas caso algum ainda não apareça, verifique se não esqueceu de incluir o atributo correto, [Fact], [Theory], [InlineData], etc antes da assinatura do método, indicando que o mesmo é um teste unitário.
- Clique no botão **Executar Todos os Testes na Exibição**, o ícone de play verde. Ou então, selecione um teste específico e execute ele individualmente.
- Nos testes que criamos, todas as estradas de dados estão dentro do esperado, ou seja, o esperado é que tenhamos sucesso em todos os testes criados.

Arquivo

Editar

Exibir

Git

Projeto

Compilação

Depurar

Teste

Ferramentas

Extensões

Janela

Ajuda

Pesquisar

AcademiaDoZe

Debug

Any CPU

AcademiaDoZe.Domain

Gerenciador de Testes

MatriculaTests.cs

LogradouroTests.cs

ColaboradorTests.cs

AlunoTests.cs

AcessoColaboradorTests.cs

AcessoAlunoTests.cs

NormalizacaoServiceTests.cs

ValueObjectsTests.cs

85

10

0

75

Pesquisar (Ctrl+I)

Execução de teste concluída: 12 Testes (12 Aprovados, 0 Com falha, 0 Ignorados) executados em 137 ms

0 Avisos0 Erros

Teste	Duração	Caracterí...	Mensagem de erro
✓ AcademiaDoZe.Domain.Tests (85)	8 ms		
> AcademiaDoZe.Domain.Tests.Entities (54)			
> AcademiaDoZe.Domain.Tests.Services (21)			
✓ AcademiaDoZe.Domain.Tests.ValueObjects (10)	8 ms		
✓ ValueObjectsTests (10)	8 ms		
> ✓ Deve_Criar_Cep_Quando_Valido (2)	< 1 ms		
> ✓ Deve_Criar_Endereco_Quando_Valido (2)	4 ms		
> ✓ Deve_Falhar_Criacao_Quando_CepDigitosInvalidos (2)	3 ms		
> ✓ Deve_Falhar_Criacao_Quando_CepNuloOuVazio (2)	< 1 ms		
> ✓ Deve_Falhar_Criacao_Quando_EnderecoInvalido (2)	1 ms		

Resumo do grupo

AcademiaDoZe.Domain.Tests.ValueObjects

Testes em grupo: 10

⌚ Duração total: 8 ms

Resultados

✓ 10 Aprovado

- Já aproveitem para incluir no mesmo arquivo, o restante dos testes de ValueObjects.

```

[Theory(DisplayName = "Cpf: nulo/vazio/espacos -> CPF_OBRIGATORIO")]
[InlineData(null)]
[InlineData("")]
[InlineData(" ")]
public void Deve_Falhar_Criacao_Quando_CpfNuloOuVazio(string? input)
{
    var result = Cpf.Criar(input!);

    Assert.True(result.IsFailure);
    Assert.Single(result.Notifications);
    Assert.Equal("CPF_OBRIGATORIO", result.Notifications.First().Mensagem);
}

[Theory(DisplayName = "Cpf: formatos válidos (com e sem pontuação)")]
[InlineData("529.982.247-25")]
[InlineData("52998224725")]
public void Deve_Criar_Cpf_Quando_ValorValido(string input)
{
    var result = Cpf.Criar(input);

    Assert.True(result.IsSuccess);
    Assert.Equal("52998224725", result.Value!.Valor);
}

[Theory(DisplayName = "Cpf: inválido - dígitos/verificador incorreto -> CPF_INVALIDO")]
[InlineData("123.456.789-00")]
[InlineData("111.111.111-11")]
public void Deve_Falhar_Criacao_Quando_CpfInvalido(string input)
{
    var result = Cpf.Criar(input);

    Assert.True(result.IsFailure);
    Assert.NotEmpty(result.Notifications);
}

[Theory(DisplayName = "Cpf: sem dígitos -> CPF_DIGITOS")]
[InlineData(" dfgdf ")]
[InlineData("abc")]
public void Deve_Falhar_Criacao_Quando_CpfSemDigitos(string input)
{
    var result = Cpf.Criar(input);

    Assert.True(result.IsFailure);
    Assert.NotEmpty(result.Notifications);
    Assert.Contains(result.Notifications, n => n.Mensagem == "CPF_DIGITOS");
}

```

```

[Theory(DisplayName = "Telefone: dígitos inválidos -> TELEFONE_DIGITOS")]
[InlineData("1234")]
[InlineData("(1)2345")]
public void Deve_Falhar_Criacao_Quando_TelefoneDigitosInvalidos(string input)
{
    var result = Telefone.Criar(input);

    Assert.True(result.IsFailure);
    Assert.NotEmpty(result.Notifications);
}

[Theory(DisplayName = "Telefone: formatos válidos (com e sem formatação)")]
[InlineData("(11) 91234-5678")]
[InlineData("11912345678")]
public void Deve_Criar_Telefone_Quando_Valido(string input)
{
    var result = Telefone.Criar(input);

    Assert.True(result.IsSuccess);
    Assert.Equal("11912345678", result.Value!.Valor);
}

[Theory(DisplayName = "Telefone: obrigatório -> TELEFONE_OBRIGATORIO")]
[InlineData(null)]
[InlineData("")]
public void Deve_Falhar_Criacao_Quando_TelefoneNuloOuVazio(string? input)
{
    var result = Telefone.Criar(input!);

    Assert.True(result.IsFailure);
    Assert.Contains(result.Notifications, n => n.Mensagem == "TELEFONE_OBRIGATORIO");
}

[Theory(DisplayName = "Senha: valida requisito de uppercase")]
[InlineData("abcdef", false)]
[InlineData("Abcdef", true)]
public void Deve_Validar_RequisitoUppercase_Senha(string senha, bool isSuccess)
{
    var result = Senha.Criar(senha);

    Assert.Equal(isSuccess, result.IsSuccess);
}

[Theory(DisplayName = "Senha: obrigatório -> SENHA_OBRIGATORIO")]
[InlineData(null)]
[InlineData("")]
public void Deve_Falhar_Criacao_Quando_SenhaNulaOuVazia(string? input)
{
    var result = Senha.Criar(input!);

    Assert.True(result.IsFailure);
    Assert.Contains(result.Notifications, n => n.Mensagem == "SENHA_OBRIGATORIO");
}

```

- Criem um novo arquivo, e nele incluam os testes para validar as normalizações.

```
using AcademiaDoZe.Domain.Services;

namespace AcademiaDoZe.Domain.Tests.Services;

public class NormalizacaoServiceTests
{
    [Theory(DisplayName = "NormalizacaoService: TextoVazioOuNulo -> valida nulo/vazio")]
    [InlineData(null, true)]
    [InlineData("", true)]
    [InlineData(" ", true)]
    [InlineData("texto", false)]
    public void Deve_TextoVazioOuNulo_RetornarEsperado(string? input, bool expected)
    {
        var result = NormalizacaoService.TextoVazioOuNulo(input);
        Assert.Equal(expected, result);
    }

    [Theory(DisplayName = "NormalizacaoService: LimparEspacos -> normaliza espaços")]
    [InlineData(null, "")]
    [InlineData("", "")]
    [InlineData(" a b c ", "a b c")]
    [InlineData("a\tb\nc", "a b c")]
    public void Deve_Normalizar_Espacos_Quando_LimparEspacosChamado(string? input, string expected)
    {
        var result = NormalizacaoService.LimparEspacos(input);
        Assert.Equal(expected, result);
    }

    [Theory(DisplayName = "NormalizacaoService: LimparTodosEspacos -> remove todos os espaços")]
    [InlineData(null, "")]
    [InlineData("", "")]
    [InlineData("a b c", "abc")]
    [InlineData(" a b ", "ab")]
    public void Deve_Remover_Todos_Espacos_Quando_LimparTodosEspacosChamado(string? input, string expected)
    {
        var result = NormalizacaoService.LimparTodosEspacos(input);
        Assert.Equal(expected, result);
    }

    [Theory(DisplayName = "NormalizacaoService: ParaMaiusculo -> converte para maiúsculo")]
    [InlineData(null, "")]
    [InlineData("", "")]
    [InlineData("abc", "ABC")]
    [InlineData("áéíôç", "ÁÉÍÔÇ")]
    public void Deve_Converter_Para_Maiusculo_Quando_ParaMaiusculoChamado(string? input, string expected)
    {
        var result = NormalizacaoService.ParaMaiusculo(input);
        Assert.Equal(expected, result);
    }

    [Theory(DisplayName = "NormalizacaoService: LimparEDigitos -> mantém apenas dígitos")]
    [InlineData(null, "")]
    [InlineData("", "")]
    [InlineData("a1b2c3", "123")]
    [InlineData("(11) 91234-5678", "11912345678")]
    [InlineData("no-digits", "")]
    public void Deve_Manter_Apenas_Digitos_Quando_LimparEDigitosChamado(string? input, string expected)
    {
        var result = NormalizacaoService.LimparEDigitos(input);
        Assert.Equal(expected, result);
    }
}
```


- Hora de validar nossos agregados raiz, ou seja, o que as outras camadas vão diretamente utilizar.
- Abuse dos testes aqui!

AcademiaDoZe\AcademiaDoZe.Domain.Tests\Entities\LogradouroTests.cs

```
using AcademiaDoZe.Domain.Entities;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class LogradouroTests
{
    [Theory(DisplayName = "Logradouro: nome vazio -> NOME_OBRIGATORIO")]
    [InlineData("")]
    [InlineData(" ")]
    public void Deve_Falhar_Criacao_Quando_NomeVazio(string nome)
    {
        var result = Logradouro.Criar(1, "12345-678", nome, "Bairro", "Cidade", "SP", "Brasil");

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "NOME_OBRIGATORIO");
    }

    [Theory(DisplayName = "Logradouro: normaliza estado removendo espaços e upper")]
    [InlineData(" s p ", "SP")]
    [InlineData(" sp ", "SP")]
    public void Deve_Normalizar_Estado_Quando_InputContemEspacos(string inputEstado, string expected)
    {
        var result = Logradouro.Criar(1, "12345-678", "Rua", "Bairro", "Cidade", inputEstado, "Brasil");

        Assert.True(result.IsSuccess);
        Assert.Equal(expected, result.Value!.Estado);
    }

    [Theory(DisplayName = "Logradouro: campos obrigatórios vazios -> mensagens específicas")]
    [InlineData("", "", "", "", "BAIRRO_OBRIGATORIO")]
    public void Deve_Falhar_Criacao_Quando_CamposObrigatoriosVazios(string rua, string bairro, string cidade, string estado, string expected)
    {
        var result = Logradouro.Criar(1, "12345-678", rua, bairro, cidade, estado, "");

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == expected);
    }
}
```

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class AlunoTests
{
    private static Logradouro GetValidLogradouro() => Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
    private static Arquivo GetValidArquivo() => Arquivo.Criar(new byte[] { 1, 2, 3 }).Value!;

    [Theory(DisplayName = "Aluno: criação bem-sucedida com nomes válidos (trim aplicado)")]
    [InlineData(" João da Silva ")]
    [InlineData("Maria")]
    public void Deve_Criar_Com_Sucesso_Quando_NomeValido(string nome)
    {
        var result = Aluno.Criar(
            1,
            nome,
            "529.982.247-25",
            DateOnly.FromDateTime(DateTime.Today.AddYears(-25)),
            "(11) 91234-5678",
            "user@example.com",
            GetValidLogradouro(),
            "123",
            "",
            "Abcdef",
            GetValidArquivo()
        );

        Assert.True(result.IsSuccess);
        Assert.Equal(nome.Trim(), result.Value!.Nome);
    }

    [Theory(DisplayName = "Aluno: nome vazio -> NOME_OBRIGATORIO")]
    [InlineData("")]
    [InlineData(" ")]
    public void Deve_Falhar_Criacao_Quando_NomeVazio(string nome)
    {
        var result = Aluno.Criar(
            1,
            nome,
            "529.982.247-25",
            DateOnly.FromDateTime(DateTime.Today.AddYears(-25)),
            "(11) 91234-5678",
            "user@example.com",
            GetValidLogradouro(),
            "123",
            "",
            "Abcdef",
            GetValidArquivo()
        );

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "NOME_OBRIGATORIO");
    }

    [Theory(DisplayName = "Aluno: data nascimento -> obrigatoriedade e idade mínima")]
    [InlineData("default", "DATA_NASCIMENTO_OBRIGATORIO")]
    [InlineData("under12", "DATA_NASCIMENTO_MINIMA_INVALIDA")]
    public void Deve_Falhar_Criacao_Quando_DataNascimentoInvalida(string scenario, string expectedMessage)
    {
        DateOnly? data = scenario == "default" ? default(DateOnly?) : DateOnly.FromDateTime(DateTime.Today.AddYears(-10));

        var dataParam = data.HasValue ? data.Value : default(DateOnly);
        var result = Aluno.Criar(
            1,
            "João",
            "529.982.247-25",
            dataParam,
            "(11) 91234-5678",
            "user@example.com",
            GetValidLogradouro(),
            "123",
            "",
            "Abcdef",
            GetValidArquivo()
        );

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == expectedMessage);
    }
}
```

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class ColaboradorTests
{
    private static Logradouro GetValidLogradouro() => Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
    private static Arquivo GetValidArquivo() => Arquivo.Criar(new byte[] { 1, 2, 3 }).Value!;

    [Theory(DisplayName = "Colaborador: data admissao obrigatória -> DATA_ADMISSAO_OBRIGATORIO")]
    [InlineData(true)]
    [InlineData(false)]
    public void Deve_Falhar_Criacao_Quando_DataAdmissaoPadrao(bool useDefault)
    {
        var dataAdmissao = useDefault ? default(DateOnly) : DateOnly.FromDateTime(DateTime.Today.AddYears(-1));

        var result = Colaborador.Criar(1, "Fulano", "529.982.247-25", DateOnly.FromDateTime(DateTime.Today.AddYears(-30)), "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", "", "Abcdef", GetValidArquivo(), dataAdmissao, ColaboradorTipo.Atendente, ColaboradorVinculo.CLT);

        if (useDefault)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "DATA_ADMISSAO_OBRIGATORIO");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "Colaborador: Administrador com vinculo inválido -> ADMINISTRADOR_CLT_INVALIDO")]
    [InlineData(ColaboradorTipo.Administrador, ColaboradorVinculo.Estagio)]
    [InlineData(ColaboradorTipo.Administrador, ColaboradorVinculo.CLT)]
    public void Deve_Falhar_Criacao_Quando_AdminComVinculoInvalido(ColaboradorTipo tipo, ColaboradorVinculo vinc)
    {
        var result = Colaborador.Criar(1, "Fulano", "529.982.247-25", DateOnly.FromDateTime(DateTime.Today.AddYears(-30)), "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", "", "Abcdef", GetValidArquivo(), DateOnly.FromDateTime(DateTime.Today.AddYears(-1)), tipo, vinc);

        if (tipo == ColaboradorTipo.Administrador && vinc == ColaboradorVinculo.Estagio)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "ADMINISTRADOR_CLT_INVALIDO");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "Colaborador: data admissao futura -> DATA_ADMISSAO_MAIOR_ATUAL")]
    [InlineData(1)]
    [InlineData(-1)]
    public void Deve_Falhar_Criacao_Quando_DataAdmissaoFutura(int daysOffset)
    {
        var date = DateOnly.FromDateTime(DateTime.Today.AddDays(daysOffset));
        var result = Colaborador.Criar(1, "Fulano", "529.982.247-25", DateOnly.FromDateTime(DateTime.Today.AddYears(-30)), "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", "", "Abcdef", GetValidArquivo(), date, ColaboradorTipo.Atendente, ColaboradorVinculo.CLT);

        if (daysOffset > 0)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "DATA_ADMISSAO_MAIOR_QUE_ATUAL");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "Colaborador: tipo ou vinculo inválido -> valida enum inválido")]
    [InlineData(999, 1)]
    [InlineData(1, 999)]
    public void Deve_Falhar_Criacao_Quando_TipoOuVinculoInvalido(int tipoValue, int vincValue)
    {
        var tipo = (ColaboradorTipo)tipoValue;
        var vinc = (ColaboradorVinculo)vincValue;
        var result = Colaborador.Criar(1, "Fulano", "529.982.247-25", DateOnly.FromDateTime(DateTime.Today.AddYears(-30)), "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", "", "Abcdef", GetValidArquivo(), DateOnly.FromDateTime(DateTime.Today.AddYears(-1)), tipo, vinc);

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
    }
}
```

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class MatriculaTests
{
    private static Logradouro GetValidLogradouro() => Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
    private static Arquivo GetValidArquivo() => Arquivo.Criar(new byte[] { 1, 2, 3 }).Value!;
    private static Aluno GetValidAluno(DateOnly? dataNascimento = null)
    {
        var nascimento = dataNascimento ?? DateOnly.FromDateTime(DateTime.Today.AddYears(-20));
        return Aluno.Criar(1, "João da Silva", "529.982.247-25", nascimento, "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", string.Empty, "Abcdef", GetValidArquivo()).Value!;
    }

    [Theory(DisplayName = "Matricula: plano inválido -> PLANO_INVALIDO; válido -> sucesso")]
    [InlineData(999)]
    [InlineData((int)MatriculaPlano.Mensal)]
    public void Deve_FalharOuPassar_Criacao_Quando_ValorDoPlano(int planoValue)
    {
        var aluno = GetValidAluno();
        var plano = (MatriculaPlano)planoValue;

        var result = Matricula.Criar(1, aluno, plano, DateOnly.FromDateTime(DateTime.Today), "Objetivo", MatriculaRestricoes.None, null);

        if (planoValue == 999)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "PLANO_INVALIDO");
        }
        else
        {
            Assert.True(result.IsSuccess);
            Assert.Equal(aluno.Id, result.Value!.AlunoId);
        }
    }

    [Theory(DisplayName = "Matricula: data inicio obrigatória -> DATA_INICIO_OBRIGATORIO")]
    [InlineData(true)]
    [InlineData(false)]
    public void Deve_Falhar_Criacao_Quando_DataInicioPadrao(bool useDefault)
    {
        var aluno = GetValidAluno();
        var inicio = useDefault ? default(DateOnly) : DateOnly.FromDateTime(DateTime.Today);

        var result = Matricula.Criar(1, aluno, MatriculaPlano.Mensal, inicio, "Objetivo", MatriculaRestricoes.None, null);

        if (useDefault)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "DATA_INICIO_OBRIGATORIO");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "Matricula: calcular DataFim por plano (mensal/trimestral/semestral/anual)*")]
    [InlineData(MatriculaPlano.Mensal, 1)]
    [InlineData(MatriculaPlano.Trimestral, 3)]
    [InlineData(MatriculaPlano.Semestral, 6)]
    [InlineData(MatriculaPlano.Anual, 12)]
    public void Deve_Calcular_DataFim_Corretamente(MatriculaPlano plano, int meses)
    {
        var aluno = GetValidAluno();
        var inicio = DateOnly.FromDateTime(DateTime.Today);

        var result = Matricula.Criar(1, aluno, plano, inicio, "Objetivo", MatriculaRestricoes.None, null);

        Assert.True(result.IsSuccess);
        Assert.Equal(inicio.AddMonths(meses), result.Value!.DataFim);
    }

    // CONTINUA
}
```

```

[Theory(DisplayName = "Matricula: restrições -> quando presentes, laudo e observações devem existir")]
[InlineData(MatriculaRestricoes.None, true)]
[InlineData(MatriculaRestricoes.Diabetes, true)]
[InlineData(MatriculaRestricoes.Diabetes | MatriculaRestricoes.Alergias, true)]
public void Deve_Tratar_Restricoes_ComOuSemLaudo(MatriculaRestricoes restricoes, bool expectSuccess)
{
    var aluno = GetValidAluno();
    Arquivo? laudo = null;
    string? observacoes = null;

    if (restricoes != MatriculaRestricoes.None)
    {
        laudo = GetValidArquivo();
        observacoes = "Observacoes";
    }

    var result = Matricula.Criar(
        1,
        aluno,
        MatriculaPlano.Mensal,
        DateOnly.FromDateTime(DateTime.Today),
        "Objetivo",
        restricoes,
        laudo,
        observacoes
    );

    Assert.Equal(expectSuccess, result.IsSuccess);
}

[Theory(DisplayName = "Matricula: menor de 16 anos exige laudo -> MENOR16_LAUDO_OBRIGATORIO")]
[InlineData(15, true)]
[InlineData(20, false)]
public void Deve_Falhar_Criacao_Quando_Menor16_ExigeLaudo(int age, bool expectFailure)
{
    var aluno = GetValidAluno(DateOnly.FromDateTime(DateTime.Today.AddYears(-age)));

    var result = Matricula.Criar(
        1,
        aluno,
        MatriculaPlano.Mensal,
        DateOnly.FromDateTime(DateTime.Today),
        "Melhorar condicionamento",
        MatriculaRestricoes.None,
        null
    );

    Assert.Equal(expectFailure, result.IsFailure);
    if (expectFailure)
    {
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "MENOR16_LAUDO_OBRIGATORIO");
    }
}

[Theory(DisplayName = "Matricula: criação e cálculo de DataFim para planos comuns")]
[InlineData(MatriculaPlano.Mensal, 1)]
[InlineData(MatriculaPlano.Trimestral, 3)]
public void Deve_Criar_Com_Sucesso_E_Calcular_DataFim(MatriculaPlano plano, int meses)
{
    var aluno = GetValidAluno();
    var inicio = DateOnly.FromDateTime(DateTime.Today);

    var result = Matricula.Criar(
        1,
        aluno,
        plano,
        inicio,
        "Melhorar condicionamento",
        MatriculaRestricoes.None,
        null
    );

    Assert.True(result.IsSuccess);
    Assert.Equal(inicio.AddMonths(meses), result.Value!.DataFim);
}

// CONTINUA

```

```

[Theory(DisplayName = "Matricula: restrições sem laudo -> RESTRICOES_LAUDO_OBRIGATORIO quando aplicável")]
[InlineData(true)]
[InlineData(false)]
public void Deve_Falhar_Criacao_Quando_RestricoesSemLaudo(bool provideLaudo)
{
    var aluno = GetValidAluno();
    Arquivo? laudo = provideLaudo ? GetValidArquivo() : null;
    var observacoes = provideLaudo ? "obs" : "";

    var result = Matricula.Criar(
        1,
        aluno,
        MatriculaPlano.Mensal,
        DateOnly.FromDateTime(DateTime.Today),
        "Objetivo",
        MatriculaRestricoes.Diabetes,
        laudo,
        observacoes
    );

    if (provideLaudo)
    {
        Assert.True(result.IsSuccess);
    }
    else
    {
        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "RESTRICOES_LAUDO_OBRIGATORIO");
    }
}

[Theory(DisplayName = "Matricula: normaliza observações de restrições removendo espaços extras")]
[InlineData(" observa  testo ", "observa testo")]
[InlineData(" obs  outro ", "obs outro")]
public void Deve_Normalizar_ObservacoesRestricoes_Quando_InputTemEspacosExtras(string input, string expected)
{
    var aluno = GetValidAluno();

    var result = Matricula.Criar(
        1,
        aluno,
        MatriculaPlano.Mensal,
        DateOnly.FromDateTime(DateTime.Today),
        "Objetivo",
        MatriculaRestricoes.Diabetes,
        GetValidArquivo(),
        input
    );

    Assert.True(result.IsSuccess);
    Assert.Equal(expected, result.Value!.ObservacoesRestricoes);
}

```

```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class AcessoAlunoTests
{
    private static Logradouro GetValidLogradouro() => Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
    private static Arquivo GetValidArquivo() => Arquivo.Criar(new byte[] { 1, 2, 3 }).Value!;
    private static Aluno GetValidAluno() => Aluno.Criar(
        1,
        "João",
        "529.982.247-25",
        DateOnly.FromDateTime(DateTime.Today.AddYears(-20)),
        "(11) 91234-5678",
        "user@example.com",
        GetValidLogradouro(),
        "123",
        string.Empty,
        "Abcdef",
        GetValidArquivo()
    ).Value!;

    [Theory(DisplayName = "AcessoAluno: aluno nulo -> ALUNO_INVALIDO")]
    [InlineData(true)]
    [InlineData(false)]
    public void Deve_Falhar_Criacao_Quando_AlunoEhNulo(bool alunoNull)
    {
        var aluno = alunoNull ? null : GetValidAluno();
        var result = AcessoAluno.Criar(1, aluno!, DateTime.Today.AddHours(10));

        if (alunoNull)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "ALUNO_INVALIDO");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "AcessoAluno: horario fora do intervalo -> DATAHORA_INTERVALO")]
    [InlineData(5, 59)]
    [InlineData(22, 1)]
    public void Deve_Falhar_Criacao_Quando_HorarioForaDoIntervalo(int hour, int minute)
    {
        var aluno = GetValidAluno();
        var dataHora = DateTime.Today.AddHours(hour).AddMinutes(minute);

        var result = AcessoAluno.Criar(1, aluno, dataHora);

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "DATA_HORA_INTERVALO_INVALIDO");
    }

    [Theory(DisplayName = "AcessoAluno: criação bem-sucedida em horários permitidos")]
    [InlineData(10)]
    [InlineData(11)]
    public void Deve_Criar_Com_Sucesso_Quando_HorarioValido(int hour)
    {
        var aluno = GetValidAluno();
        var dataHora = DateTime.Today.AddHours(hour); // 10:00 or 11:00

        var result = AcessoAluno.Criar(1, aluno, dataHora);

        Assert.True(result.IsSuccess);
        Assert.Equal(aluno.Id, result.Value!.AlunoId);
        Assert.Equal(dataHora, result.Value.DataHora);
    }

    [Theory(DisplayName = "AcessoAluno: permite horários de borda 06:00 e 22:00")]
    [InlineData(6)]
    [InlineData(22)]
    public void Deve_Permittir_HorariosDeBorda_06_00_e_22_00(int hour)
    {
        var aluno = GetValidAluno();
        var time = DateTime.Today.AddHours(hour);

        var r = AcessoAluno.Criar(1, aluno, time);

        Assert.True(r.IsSuccess);
    }
}
```



```
using AcademiaDoZe.Domain.Entities;
using AcademiaDoZe.Domain.Enums;
using AcademiaDoZe.Domain.ValueObjects;

namespace AcademiaDoZe.Domain.Tests.Entities;

public class AcessoColaboradorTests
{
    private static Logradouro GetValidLogradouro() => Logradouro.Criar(1, "12345-678", "Rua Teste", "Bairro", "Cidade", "SP", "Brasil").Value!;
    private static Arquivo GetValidArquivo() => Arquivo.Criar(new byte[] { 1, 2, 3 }).Value!;
    private static Colaborador GetValidColaborador() => Colaborador.Criar(1, "Fulano", "529.982.247-25", DateOnly.FromDateTime(DateTime.Today.AddYears(-30)), "(11) 91234-5678", "user@example.com", GetValidLogradouro(), "123", string.Empty, "Abcdef", GetValidArquivo(),
        DateOnly.FromDateTime(DateTime.Today.AddYears(-1)),
        ColaboradorTipo.Atendente,
        ColaboradorVinculo.CLT
    ).Value!;

    [Theory(DisplayName = "AcessoColaborador: colaborador nulo -> COLABORADOR_INVALIDO")]
    [InlineData(true)]
    [InlineData(false)]
    public void Deve_Falhar_Criacao_Quando_ColaboradorEhNulo(bool colaboradorNull)
    {
        var colaborador = colaboradorNull ? null : GetValidColaborador();
        var result = AcessoColaborador.Criar(1, colaborador!, DateTime.Today.AddHours(10));

        if (colaboradorNull)
        {
            Assert.True(result.IsFailure);
            Assert.NotEmpty(result.Notifications);
            Assert.Contains(result.Notifications, n => n.Mensagem == "COLABORADOR_INVALIDO");
        }
        else
        {
            Assert.True(result.IsSuccess);
        }
    }

    [Theory(DisplayName = "AcessoColaborador: horario fora do intervalo -> DATAHORA_INTERVALO")]
    [InlineData(5, 59)]
    [InlineData(22, 1)]
    public void Deve_Falhar_Criacao_Quando_HorarioForaDoIntervalo(int hour, int minute)
    {
        var colaborador = GetValidColaborador();
        var dataHora = DateTime.Today.AddHours(hour).AddMinutes(minute);

        var result = AcessoColaborador.Criar(1, colaborador, dataHora);

        Assert.True(result.IsFailure);
        Assert.NotEmpty(result.Notifications);
        Assert.Contains(result.Notifications, n => n.Mensagem == "DATA_HORA_INTERVALO_INVALIDO");
    }

    [Theory(DisplayName = "AcessoColaborador: criação bem-sucedida em horários permitidos")]
    [InlineData(10)]
    [InlineData(14)]
    public void Deve_Criar_Com_Sucesso_Quando_HorarioValido(int hour)
    {
        var colaborador = GetValidColaborador();
        var dataHora = DateTime.Today.AddHours(hour); // 10:00 or 14:00

        var result = AcessoColaborador.Criar(1, colaborador, dataHora);

        Assert.True(result.IsSuccess);
        Assert.Equal(colaborador.Id, result.Value!.ColaboradorId);
        Assert.Equal(dataHora, result.Value.DataHora);
    }

    [Theory(DisplayName = "AcessoColaborador: permite horários de borda 06:00 e 22:00")]
    [InlineData(6)]
    [InlineData(22)]
    public void Deve_Permittir_HorariosDeBorda_06_00_e_22_00(int hour)
    {
        var colaborador = GetValidColaborador();
        var time = DateTime.Today.AddHours(hour);

        var r = AcessoColaborador.Criar(1, colaborador, time);

        Assert.True(r.IsSuccess);
    }
}
```

The screenshot displays the Visual Studio test runner interface. At the top, there's a toolbar with various icons for test execution and debugging. Below it, a status bar indicates the overall test execution status: "Execução de teste concluída: 112 Testes (110 Aprovados, 2 Com falha, 0 Ignorados) executados em 181 ms".

The main area is divided into two panes. On the left, a tree view lists the test hierarchy:

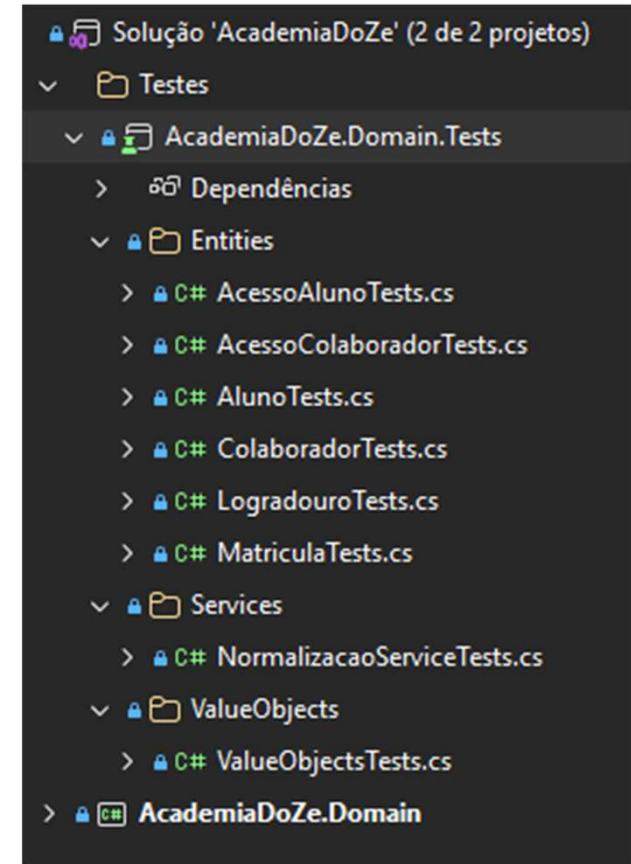
- AcademiaDoZe.Domain.Tests (112)**: Total duration 51 ms.
 - AcademiaDoZe.Domain.Tests.Entities (54)**: 43 ms.
 - AcademiaDoZe.Domain.Tests.Services (21)**: 4 ms.
 - AcademiaDoZe.Domain.Tests.ValueObjects (37)**: 4 ms.
 - ValueObjectsTests (37)**: 4 ms.
 - Deve_Criar_Arquivo_Quando_Valido (2)**: < 1 ms.
 - Deve_Criar_Cep_Quando_Valido (2)**: < 1 ms.
 - Deve_Criar_Cpf_Quando_valorValido (2)**: < 1 ms.
 - Deve_Criar_Email_E_RemoverEspacos_Quando_InputTemEspacos (2)**: < 1 ms.
 - Deve_Criar_Endereco_Quando_Valido (2)**: < 1 ms.
 - Deve_Criar_Telefone_Quando_Valido (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_CepDigitosInvalidos (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_CepNuloOuVazio (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_CpfInvalido (2)**: 4 ms.
 - Cpf: inválido - dígitos/verificador incorreto -> CPF_INVALIDO(input: "..."): Assert.True() Failure Expected: True Act...
 - Cpf: inválido - dígitos/verificador incorreto -> CPF_INVALIDO(input: "..."): Assert.True() Failure Expected: True Act...
 - Deve_Falhar_Criacao_Quando_CpfNuloOuVazio (3)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_CpfSemDigitos (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_EnderecoInvalido (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_FormatoEmailInvalido (4)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_SenhaNulaOuVazia (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_TelefoneDigitosInvalidos (2)**: < 1 ms.
 - Deve_Falhar_Criacao_Quando_TelefoneNuloOuVazio (2)**: < 1 ms.
 - Deve_Validar_RequisitoUppercase_Senha (2)**: < 1 ms.

On the right side, the "Summary of group" pane provides a high-level overview:

- Resumo do grupo**: AcademiaDoZe.Domain.Tests
- Testes em grupo: 112
- Duração total: 51 ms
- Resultados**:
 - 110 Aprovado
 - 2 Falha

- O padrão **AAA (Arrange, Act, Assert)** é uma abordagem para organizar testes unitários de forma clara e compreensível:
 - **Arrange** (Organizar)
 - Prepare os dados, objetos e dependências necessários para o teste.
 - **Act** (Agir)
 - Execute a ação que você deseja testar (ex: chamar um método).
 - **Assert** (Verificar)
 - Verifique se o resultado está conforme o esperado (ex: usando asserts para comparar valores ou verificar exceções).
- Esse padrão facilita a leitura, manutenção e entendimento dos testes, tornando claro cada etapa do processo de validação.

- Perfeito!
- Com essa base inicial de conhecimento sobre testes unitários, vocês já podem criar testes para validar toda nossa camada de domínio.
- A medida que formos criando novas camadas, precisaremos ver outras formas e recursos de testes, mas isso é algo que nos preocuparemos na hora apropriada.



Laboratório prático

Parcial de nota avaliação 02

- Complemente a implementação da sua camada de domínio, compatibilizando ela com a exemplificação.
- Para uso futuro, já realize a implementação dos repositórios.
- Crie e realize **testes unitários** de toda sua camada de domínio.
- Tudo, sem exceção, deve ser amplamente testado! Isso é fundamental para não ter problemas futuros!

Laboratório prático

Parcial de nota avaliação 02

- Data de entrega:
 - Turma I: 10/08
 - Turma II: 13/08
- Data de entrega como recuperação:
 - Turma I: 17/08 18:40
 - Turma II: 20/08 18:40
- **Print do gerenciador da solução aberto, com todos os diretórios expandidos, incluindo o projeto da camada de domínio e dos testes.**
- **Print com a o gerenciador de testes** aberto, devendo possuir no mínimo 120 testes executados com sucesso.
 - Não pode ser compactado.
 - Na primeira linha de cada arquivo, deve conter um comentário com seu nome.